ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл, холбооны технологийн сургууль



БИЕ ДААЛТ-1
ГАРЫН АВЛАГА

Үйлдийн систем IV-2-4

[F.CSM302]

2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр:	**Kernel Data Structures**
Бие даалтын ажлыг гүйцэтгэсэн оюутан:	**Б. Лхамрагчаа /B241940102/** **Г. Үлэмж /B232270805/** **Т. Буянтөгс /B231910034/** **Э. Төгсбаяр /221910021/** **Ө. Чинзориг /200900028/** **Б. Идэрбат /B242270075/**
Хичээл заасан багш :	**Г.ГАНБАТ /F.CS10/**

Улаанбаатар 2026 он

Гарчиг

1 Хэрэглэгчийн заавар....3
1.1 Програмын зорилго....3
1.2 Дэмжих орчин....3
1.3 Суулгах, нээх алхам....3
1.4 Оролт ба тохиргоо....3
1.5 Ажиллуулах, зогсоох, reset хийх....4
1.6 Дүрслэлийн тэмдэг, өнгө, төлөв....5
1.7 Жишээнүүд....5
Жишээ 1 — Номын Part I: 4 өнгө, хайх, устгах....5
Жишээ 2 — Part II: Collatz дараалал, хайх, алдаа....6
Жишээ 3 — Стек (LIFO) ба дараалал (FIFO)....7
Жишээ 4 — Командыг URL-ээр ажиллуулах....7
Жишээ 5 — ENOMEM тарилга....7
1.8 Түгээмэл алдаа ба шийдэл....8
2 Operational заавар....8
2.1 Repository clone хийх....8
2.2 OS, compiler/runtime, хувилбар....9
2.3 Dependency суулгах....9
2.4 Build ба Run командууд....9
2.5 Үндсэн тестийн өгөгдөл....10
2.6 Нэмэлт boundary/error тест ба хүлээгдсэн үр дүн....10
2.7 Жинхэнэ кернел модуль (суулгах, ачаалах, зайлуулах)....11
2.8 Хүлээгдсэн `make test` үр дүн....12
2.9 Зарим засварлах дутагдлууд....13

1 Хэрэглэгчийн заавар

Энэ хэсэгт программыг хэрхэн суулгаж ажиллуулах, үндсэн тохиргоонуудыг өөрчлөх, графикийг уншиж ойлгох болон түгээмэл алдаа гарсан үед хэрхэн засах талаар тайлбарлана. Доорх командуудыг төслийн үндсэн хавтас дотроос ажиллуулна.

Хурдан эхлэх: Эхлээд терминал дээр `make` командыг ажиллуулж программыг хөрвүүлнэ. Дараа нь `make test` командаар тестүүдийг ажиллуулж шалгана. Бүх зүйл хэвийн бол `make run` командыг ажиллуулж серверийг асаана.

Сервер ассан дараа браузераа нээгээд `http://127.0.0.1:8080` хаяг руу орсноор программыг ашиглаж эхэлнэ.

1.1 Програмын зорилго

Энэхүү програм нь Linux kernel-ийн дугуй хоёр холбоост жагсаалт болох `struct list_head`-ийн ажиллагааг туршиж, ойлгоход зориулсан. Жагсаалт дээр үйлдэл хийх бүрд `next` болон `prev` заагчууд хэрхэн өөрчлөгдөж байгаа, мөн `kmalloc` болон `kfree` ашиглан санах ой хэрхэн хуваарилагдаж, чөлөөлөгдөж байгааг графикаар шууд харуулна.

- **Бодит өгөгдөл дээр ажиллана:** UI дээр харагдаж байгаа график нь урьдчилан бэлдсэн mockup биш. C engine доторх жагсаалтын бодит төлөв болон заагчийн мэдээлэл дээр үндэслэн зурагддаг.
- **Дэмжигдэх бүтэцүүд:**
  - Өнгөний жагсаалт (Part I)
  - Collatz дараалал (Part II)
  - Стек (`Stack`)
  - Дараалал (`Queue`)

1.2 Дэмжих орчин

- **Үйлдлийн систем:** Программыг Linux орчинд ажиллуулахаар хийсэн. Arch Linux, Linux 7.2.11 дээр туршиж шалгасан.
  Анхааруулга: Программ нь Linux-ийн `/proc` файлын системийг ашигладаг тул macOS болон Windows дээр шууд ажиллахгүй.
- **Шаардлагатай хэрэгслүүд:**
  - `gcc` — 16.2.1 хувилбараар туршсан
  - `make` — GNU Make 4.4.1
  - `python3` — 3.14.8 хувилбараар туршсан бөгөөд нэмэлт сан суулгах шаардлагагүй, зөвхөн стандарт санг ашиглана
- **Браузер:** JavaScript болон SVG дүрслэл дэмждэг орчин үеийн браузер байхад хангалттай. Жишээлбэл Brave, Chromium зэрэг браузер ашиглаж болно.

Программыг ажиллуулахад интернет холболт шаардлагагүй.

1.3 Суулгах, нээх алхам

Программыг ажиллуулахдаа дараах дарааллыг ашиглана.

1. Эхлээд repository-г татаж авна:
   `git clone <GitHub холбоос> kernel-data-structures && cd kernel-data-structures`
   Хэрэв GitHub холбоос байхгүй бол хүргүүлсэн төслийн хавтсыг компьютертээ хуулж,

терминалаас тухайн хавтас руу `cd` командаар орно. Үүнийг 8.2.1 хэсэгт дэлгэрэнгүй тайлбарласан.

2. Дараа нь:
   ```
   make
   ```
   командыг ажиллуулна. Амжилттай дууссаны дараа `build/kds_engine` файл үүснэ.
3. Хүсвэл тестүүдийг:
   ```
   make test
   ```
   командаар ажиллуулж шалгаж болно. Бүх тест амжилттай дуусвал хамгийн сүүлд `All available tests passed.` гэсэн мэдээлэл гарна.
4. Серверийг асаахдаа:
   ```
   make run
   ```
   командыг ажиллуулна. Терминал дээр `http://127.0.0.1:8080` гэсэн хаяг гарч ирнэ.
5. Уг хаягийг браузераар нээнэ. Хэрэв дэлгэцийн дээд хэсэгт **«Хөдөлгүүр холбогдсон»** гэсэн төлөв харагдаж байвал програм ашиглахад бэлэн болсон гэсэн үг.

1.4 Оролт ба тохиргоо

Хүснэгт 1. UI-ийн үйлдлүүд ба оролтын хязгаар

Таб	Үйлдэл (UI дээрх нэр)	Оролт
Өнгөний жагсаалт	Төгсгөлд нэмэх / Эхэнд нэмэх (`add_tail`/`add_head`)	`red`, `green`, `blue`: 0…255
	Индексийн дараа оруулах (`insert`)	`index` (0-ээс; тэр индекстэй зангилааны **дараа**) + өнгө
	Индексээр устгах	`index`
	Өнгөөр устгах / Өнгөөр хайх	Дор хаяж нэг өнгө; тохирох **бүх** зангилаа
	Жагсаалтыг гүйлгэх / хооослох	—
Collatz	Дараалал үүсгэх (`run`)	`start`: 1…2147483647 (өмнөхийг солино)
	Утгаар хайх / устгах; гүйлгэх; хооослох	`value`: бүхэл тоо
Стек	push (толгойд), pop, peek	`value`: тэмдэгтэй 64 бит
Дараалал	enqueue (төгсгөлд), dequeue, peek	`value`: тэмдэгтэй 64 бит

Бэлэн жишээ: Програм дээр шууд ажиллуулж үзэх хоёр бэлэн жишээ байгаа. Үүнд **«Номын 4 өнгө»** (`color example`) болон **«Collatz · 27-оос эхлэх»** жишээ багтана.

Командыг шууд оруулах: UI доторх «Командыг шууд оруулах» хэсэгт engine-д өгөх командыг гараар бичиж болно. Жишээлбэл `inject_fail after=2` эсвэл `check` зэрэг команд ашиглаж болно. Нэг команд нэг мөрөнд бичигдэх бөгөөд 512 тэмдэгтээс ихгүй байна.

Серверийн тохиргоо: Серверийг `python3 src/ui/server.py` командаар шууд ажиллуулах үед хэд хэдэн параметр өгч болно.

- `--port` — серверийн порт. Анхдагч утга нь `8080`.

- `--max-nodes` — нэг бүтэц дээр ашиглаж болох зангилааны дээд хязгаар. `1–100000` мужид утга авна, анхдагч нь `2048`.
- `--engine` — ашиглах engine-ийн зам. Анхдагч нь `build/kds_engine`.

Жишээлбэл:

```
python3 src/ui/server.py --port 9000 --max-nodes 50
```

гэсэн командаар серверийг 9000 порт дээр ажиллуулж, нэг бүтэц дээр хамгийн ихдээ 50 зангилаа ашиглахаар тохируулж болно.

URL параметрүүд: Browser-ийн URL дээр нэмэлт параметр өгч UI-г урьдчилан тохируулж болно. `?run=cmd1;cmd2` ашиглавал командууд дараалаар автоматаар ажиллана. `?tab=color|collatz|stack|queue` параметрээр эхлээд нээгдэх табыг сонгож, `?theme=dark` ашиглан dark theme-ийг идэвхжүүлж болно.

1.5 Ажиллуулах, зогсоох, reset хийх

- **Ажиллуулах:** Программыг `make run` командаар асаана. UI дээр үйлдэл хийхдээ **«Үйлдлийг гүйцэтгэх»** товчийг дарах эсвэл `Enter` ашиглаж болно.
- **Зогсоох:** `make run` ажиллаж байгаа терминал дээр `Ctrl+C` дарж серверийг зогсооно. Engine хаагдах үед санах ойд чөлөөлөгдөөгүй зангилаа үлдсэн эсэхийг шалгана. Хэрэв үлдсэн бол `LEAK: live=N` гэсэн мэдээлэл гарна. Хэвийн ажиллагааны үед энэ мэдээлэл гарахгүй.
- **Reset хийх: «Бүх бүтцийг хоослох»** товч буюу `reset` командаар өнгө, Collatz, стек, дараалал гэсэн 4 бүтцийн бүх зангилааг чөлөөлнө. Мөн өмнө тохируулсан санах ойн алдаа үүсгэх горимыг унтраана. Харин `kmalloc` болон `kfree`-ийн нийт хуримтлагдсан тоолуур тэглэгдэхгүй.
- **Engine-ийг бүрэн дахин эхлүүлэх:** Серверийг зогсоогоод дахин `make run` ажиллуулж болно. Мөн сервер ажиллаж байгаа үед `curl -X POST http://127.0.0.1:8080/api/restart` командаар engine-ийг дахин эхлүүлэх боломжтой.
- **Үйлдлийн түүх: «Үйлдлийн түүх»** хэсгийн ← болон → товч, гулсуур эсвэл **«Сүүлийн алхам»** товчоор өмнөх төлвүүдийг үзэж болно. Өмнөх алхмыг үзэж байх үед зөвхөн харах боломжтой бөгөөд жагсаалтыг өөрчлөх үйлдлүүд түр түгжигдэнэ. Түүх нь тухайн browser-ийн таб дотор хадгалагдах тул хуудсыг refresh хийвэл арилна. Харин engine доторх одоогийн бүтэц хэвээр үлдэнэ.
- **Анхаарах зүйл:** Хэрэв URL-д `?run=...` параметр ашигласан бол хуудсыг дахин ачаалах бүрд тухайн командууд дахин ажиллана.
-

1.6 Дүрслэлийн тэмдэг, өнгө, төлөв

Хүснэгт 2. Графын тайлбар

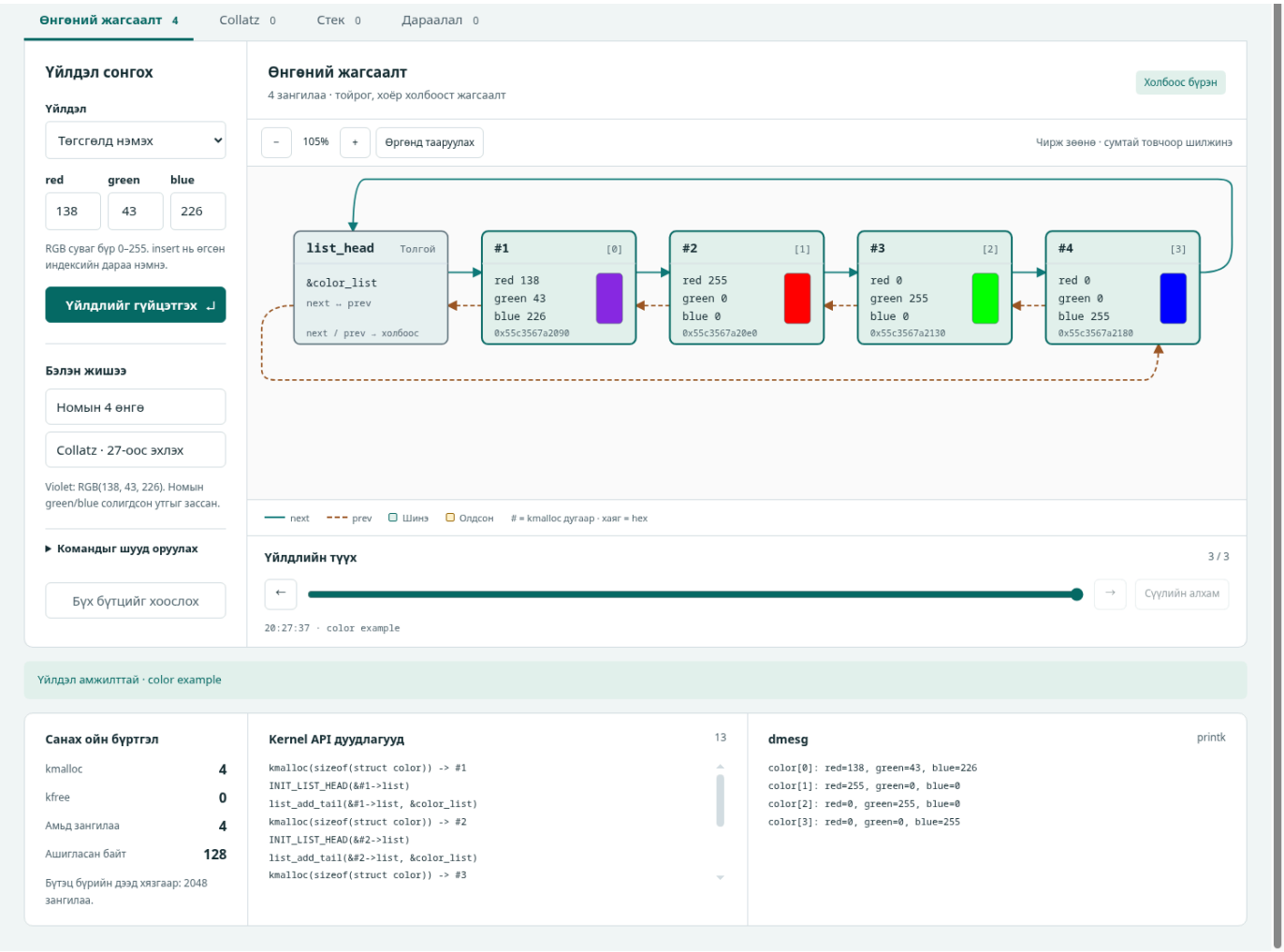
Тэмдэг	Утга
list_head (саарал, «Толгой»)	Жагсаалтын толгой (`&color_list` гэх мэт). `next`/`prev` нь анхны ба сүүлийн зангилааг заана.
Зангилааны хайрцаг `#7 [0]`	`#` нь `kmalloc`-ийн дугаар (serial). `[ ]` нь жагсаалт дахь индекс (0-ээс). Дор нь талбарууд ба хаяг (hex).
Өнгөний swatch	`rgb(red, green, blue)`-ийн бодит өнгө.
Тасралтгүй сум (**next**)	Зангилааны `next` заагч. Сүүлийн зангилаанаас толгой руу **нум** байвал бүтэц дугуй (circular).
Тасархай сум (**prev**)	Зангилааны `prev` заагч. Толгойноос сүүлийн зангилаа руу нум.
Улаан `?`	Заагч бүртгэлд байхгүй зангилаа руу заасан. Жагсаалт эвдэрсэн.
Толгойн өөртөө заасан сум	Жагсаалт хоосон (`head->next == head`).
TOP · орой, **FRONT · эхлэл**, **REAR · төгсгөл**	Стек ба дарааллын байрлалын шошго.
«Шинэ» (ногоон хүрээ)	Сая нэмэгдсэн зангилаа.
«Олдсон» (шар хүрээ)	Хайлтаар олдсон зангилаа. Хайлтын явцад гүйлгэсэн зангилаанууд дараалаар тодорно (анимаци).
Тэмдэг «Холбоос бүрэн» / «Холбоос алдаатай»	`kds_verify`-ийн үр дүн (`next→prev`, `prev→next`, хоёр чиглэлийн тоо).
Санах ойн бүртгэл	`kmalloc` (нийт дуудсан), `kfree`, *Бүтэн зангилаа* = `kmalloc − kfree`, *Ашигласан байт*.
Kernel API дуудлагууд / dmesg	Үйлдлийн бодит `kmalloc`/`list_*`/`kfree` дуудлага; `printk`-ийн мөр. (UI дээрх хэсгийн нэр.)
Дээд талын төлөв	«Хөдөлгүүр холбогдсон» / «Холболтын алдаа». Улаан мөр нь алдаа (код + тайлбар).
Томруулах − 105 % +, «Өргөнд тааруулах»	Том жагсаалтад (Collatz 27 = 112 зангилаа) ойртуулах/холдуулах. Графыг чирж зөөнө; сумтай товчоор шилжинэ.
«Бараан/Цайвар горим»	Өнгөний горим солих.

1.7 Жишээнүүд

Жишээ 1 — Номын Part I: 4 өнгө, хайх, устгах

1. **Өнгөний жагсаалт** таб руу орж **«Номын 4 өнгө»** товчийг дарна. Үүний дараа нийт 4 зангилаа үүснэ. `dmesg` хэсэгт дараах утгууд харагдана:
   ```
   color[0]: red=138, green=43, blue=226
   color[1]: red=255, green=0, blue=0
   color[2]: red=0, green=255, blue=0
   color[3]: red=0, green=0, blue=255
   ```
   Хэрэв engine-ийг дөнгөж асаасан бол санах ойн тоолуур `kmalloc = 4`, `kfree = 0`, идэвхтэй зангилаа `4`, нийт ашиглаж буй санах ой `128 байт` байна. Эхний зангилаа нь violet өнгө бөгөөд RGB утга нь `(138, 43, 226)` байна.
2. Дараа нь **«Өнгөөр хайх»** үйлдлийг сонгож, `red = 255` гэж өгнө. Хайлтын үр дүнд хоёр дахь зангилаа буюу индекс `1` дээрх элемент **«Олдсон»** гэж тодорч харагдана. Мөн хайлт ямар дараалаар явагдсаныг `#1 → #2` гэсэн анимацаар үзүүлнэ.

3. Үүний дараа **«Индексээр устгах»** үйлдэл дээр `index = 1` гэж өгнө. Энэ үед хоёр дахь зангилааг `list_del(&#2->list)` ашиглан жагсаалтаас салгаж, `kfree(#2)`-оор санах ойг нь чөлөөлнө. График шинэчлэгдэж `#1 → #3 → #4` гэсэн дараалалтай болно. Санах ойн тоолуур энэ үед `kmalloc = 4`, `kfree = 1`, идэвхтэй зангилаа `3`, нийт санах ой `96 байт` болно.
4. Эцэст нь **«Бүх бүтцийг хоослох»** товчийг дарна. Ингэснээр үлдсэн бүх зангилаа чөлөөлөгдөж, идэвхтэй зангилааны тоо `0` болно.

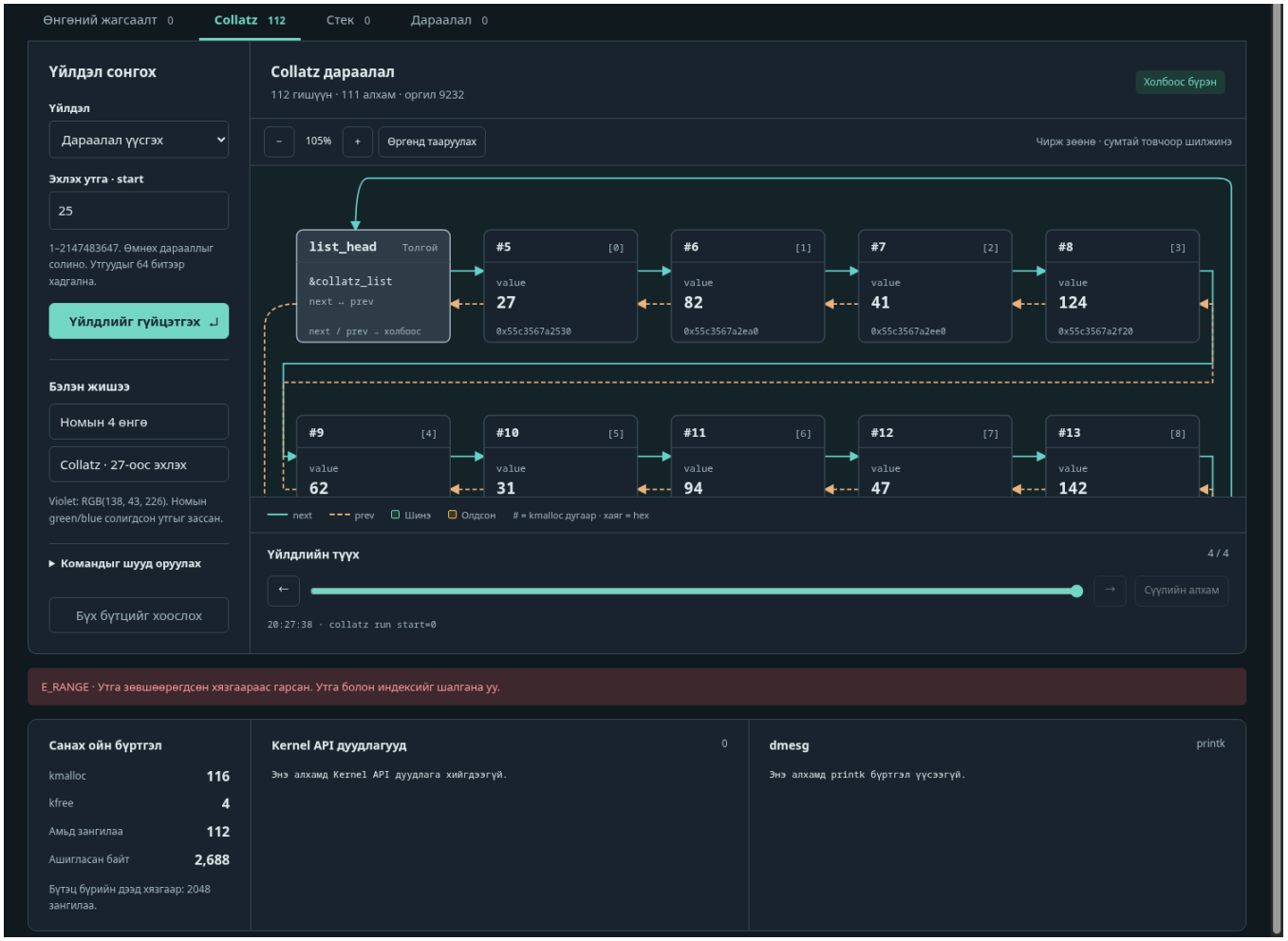


Жишээ 1, алхам 1: «Номын 4 өнгө».

Жишээ 2 — Part II: Collatz дараалал, хайх, алдаа

1. **Collatz** таб руу орж **«Дараалал үүсгэх»** үйлдлийг сонгоод `start = 27` гэж өгнө. Үүний дараа нийт 112 зангилаа, 111 алхамтай Collatz дараалал үүснэ. Хамгийн их утга нь `9232` байна. Дарааллын эхлэл нь `27 → 82 → 41 → 124 → 62 → 31 → ...` байдлаар үргэлжилнэ. Энэ үед идэвхтэй зангилааны тоо 112 бөгөөд нэг зангилаа 24 байт тул нийт `2,688 байт` санах ой ашиглана.
2. Дараа нь **«Утгаар хайх»** үйлдлийг сонгож `value = 9232` гэж өгнө. Хайлт жагсаалтын эхнээс явагдаж, хамгийн их утга болох `9232` нь 77-р индекс дээр олдоно.

3. **Алдааны шалгалт: «Командыг шууд оруулах»** хэсэгт `collatz run start=0` гэж бичиж илгээнэ. Энэ шалгалтыг формоор хийх боломжгүй. Учир нь **«Дараалал үүсгэх»** формын `start` талбар `min=1` гэсэн хязгаартай тул browser `0` утгыг engine рүү явуулахаас өмнө өөрөө хаана. Харин командыг шууд өгөхөд engine `E_RANGE` алдаа буцаана. Энэ үед өмнөх 112 зангилаатай граф болон санах ойн тоолуур өөрчлөгдөхгүй хэвээр үлдэнэ.
4. Үүний дараа **«Утгаар устгах»** үйлдлийг сонгож `value = 9232` гэж өгнө. Тухайн зангилаа устсаны дараа жагсаалтад 111 зангилаа үлдэнэ.
5. Дараа нь **«Дараалал үүсгэх»** үйлдэл дээр `start = 837799` гэж өгч дахин ажиллуулна. Энэ тохиолдолд нийт 525 зангилаа үүсэж, хамгийн их утга нь `2,974,984,576` хүрнэ. Энэ нь 32 битийн тэмдэгтэй бүхэл тооны дээд хязгаараас их тул Collatz-ийн утгуудыг 64 битийн төрлөөр хадгалсан.



Жишээ 2, алхам 3: `collatz run start=0` → E_RANGE; 112 зангилаатай төлөв өөрчлөгдөөгүй

Жишээ 3 — Стек (LIFO) ба дараалал (FIFO)

Стек: Эхлээд `push 10`, дараа нь `push 42`, эцэст нь `push -2` үйлдлүүдийг дарааллаар хийсэн. Ингэснээр стек дээр хамгийн сүүлд нэмэгдсэн утга болох `-2` дээд талд байрлаж, `TOP = -2` болсон. Дараа нь `pop` хийхэд `-2` утга гарч, стек дээрх шинэ дээд утга `42` болж `TOP = 42` болсон.

Дараалал: Эхлээд `enqueue 10`, дараа нь `enqueue 42` гэж нэмсэн. Ингэснээр дарааллын эхний утга `10`, хамгийн сүүлд нэмэгдсэн утга `42` болж, `FRONT = 10`, `REAR = 42` болсон. Дараа нь `dequeue` хийхэд хамгийн эхэнд байсан `10` утга гарч, дараалалд `42` үлдсэн.

Жишээ 4 — Командыг URL-ээр ажиллуулах

Жишээлбэл дараах URL-ийг нээж болно:

```
http://127.0.0.1:8080/?run=reset;color%20example;color%20delete%20index=1
```

Энэ URL нээгдэхэд командууд дарааллаар автоматаар ажиллана. Эхлээд `reset` хийгдэж өмнөх өгөгдлүүд цэвэрлэгдэнэ. Дараа нь `color example` командаар номын 4 өнгийн жишээ үүснэ. Эцэст нь `color delete index=1` командаар 1-р индекс дээр байгаа өнгийн зангилаа устгагдана.

Жишээ 5 — ENOMEM тарилга

«Командыг шууд оруулах» хэсэгт эхлээд `inject_fail after=2` командыг өгнө. Энэ тохиргоо нь хоёр амжилттай allocation хийсний дараагийн `kmalloc` дээр зориудаар алдаа үүсгэнэ.

Дараа нь `color add_tail red=1 green=2 blue=3` командыг нийт гурван удаа ажиллуулна. Эхний хоёр удаагийн нэмэлт амжилттай хийгдэнэ. Харин гурав дахь удаад санах ой хуваарилж чадахгүй тул engine `E_NOMEM` алдаа болон `kmalloc failed; the previous state is preserved` гэсэн тайлбар буцаана. Энэ үед өмнөх жагсаалтын граф болон төлөв өөрчлөгдөхгүй хэвээр үлдэнэ.

Туршилт дууссаны дараа `inject_fail off` командыг өгч санах ойн алдааг зориудаар үүсгэх тохиргоог унтраана.

1.8 Түгээмэл алдаа ба шийдэл

Хүснэгт 3. Түгээмэл алдаа

Харагдах зүйл	Шалтгаан	Шийдэл
Хуудас нээгдэхгүй; «Холболтын алдаа»	Сервер ажиллахгүй, эсвэл өөр порт	`make run` ажиллаж байгаа эсэх, терминал дахь хаягийг шалгах
`Address already in use`	8080 порт завгүй	`python3 src/ui/server.py --port 8081` (эсвэл өөр процессыг зогсоох)
`build/kds_engine` олдохгүй; `make run` унана	Бүтээгээгүй	Эхлээд `make`
`cc: command not found`, `make: command not found`, `python3: command not found`	Хэрэгсэл суулгаагүй	gcc, make, python3 суулгах (8.2.3)

Харагдах зүйл	Шалтгаан	Шийдэл
E_RANGE (өнгө 256, −1; index; start 0, 2147483648)	Хязгаарааас хэтэрсэн. Формын `min`/`max` нь ихэнхийг browser дээр нь хаадаг. «Командыг шууд оруулах» эсвэл HTTP-ээр шууд илгээвэл engine **E_RANGE** буцаана.	Өнгө 0…255, `start` 1…2147483647, индекс 0…(тоо−1)
E_BAD_VALUE	Тоо биш утга (`abc`)	Аравтын бүхэл тоо оруулах
E_MISSING_ARG / **E_UNKNOWN_ARG** / **E_SYNTAX**	Дутуу, мэдэхгүй, давхардсан аргумент, эсвэл 512-аас урт мөр	Командыг `key=value` хэлбэрээр, нэг түлхүүрийг нэг удаа бичих
E_EMPTY	Хоосон стек/дарааллаас pop/dequeue/peek	Эхлээд push/enqueue хийх
E_NOT_FOUND	Хайлт/устгал тохирохгүй	Хэвийн; өөр утга өгөх
E_FULL	`--max-nodes` хязгаар дүүрсэн	Хоослох/reset, эсвэл `--max-nodes`-ийг нэмэх
E_NOMEM	Тарьсан санах ойн алдаа (`inject_fail`)	`inject_fail off` эсвэл «Бүх бүтцийг хоослох»
Үйлдлийн товч идэвхгүй; «Түүхийн агшин — зөвхөн уншина»	Өнгөрсөн алхмыг үзэж байна	«Сүүлийн алхам» товч
Хуучин дүрслэл харагдана	Browser-ийн кэш	Хуудсыг шахан дахин ачаалах (`Ctrl+Shift+R`)

2 Operational заавар

2.1 Repository clone хийх

Хэрэв GitHub холбоос байгаа бол төслийг дараах командаар татаж авна:

```
git clone <GitHub холбоос> kernel-data-structures
```

Дараа нь төслийн хавтас руу орно:

```
cd kernel-data-structures
```

Хэрэв GitHub холбоос ашиглах боломжгүй, оффлайн орчинд ажиллаж байгаа бол хүргэж өгсөн `kernel-data-structures` хавтас эсвэл архивыг өөрийн компьютерт хуулна. Архив хэлбэрээр өгөгдсөн бол эхлээд задлаад, дараа нь тухайн хавтас руу орно. Үүнээс хойших алхмууд GitHub-оос татсан тохиолдолтой ижил байна.

Жишээ нь:

```
cp -r /зам/kernel-data-structures ~/kernel-data-structures
```

эсвэл архив байвал:

```
tar -xf kernel-data-structures.tar
```

Дараа нь:

```
cd ~/kernel-data-structures
```

Төслийн үндсэн хавтас дотор `README.md`, `Makefile`, `src/`, `tests/`, `docs/` файлууд болон хавтаснууд байна. `src/` дотор `shim/`, `kernel/`, `engine/`, `ui/` гэсэн үндсэн хэсгүүд байрлана.

2.2 OS, compiler/runtime, хувилбар

Зүйл	Туршсан хувилбар	Тайлбар
OS ба кернел	Arch Linux, Linux 7.2.11-arch1-1 x86_64	Өөр Linux тархалтад туршаагүй
C compiler	gcc (GCC) 16.2.1 20260810	GNU C11 (`-std=gnu11`), `__typeof__` ашигладаг; ASan/UBSan дэмжих ёстой
make	GNU Make 4.4.1	
python3	3.14.8	Зөвхөн стандарт сан
Browser	Brave 156.1.98.62 (Chromium)	Зөвхөн Chromium-д туршсан. Screenshot-ыг өмнөх 156.1.98.60-аар авсан.
Кернелийн header	linux-headers 7.2.11.arch1-1	Зөвхөн `.ko` module бүтээхэд (8.2.7). `make test` хэрэглэхгүй.

Хамгийн бага шаардлагатай хувилбарыг тодорхойлоогүй (зөвхөн дээрх хувилбараар туршсан).

2.3 Dependency суулгах

Програм нь гуравдагч талын нэмэлт сан шаардахгүй. Хэрэв `gcc`, `make`, `python3` зэрэг хэрэгслүүд суулгагдаагүй бол доорх командуудыг ашиглаж болно. Эдгээр нь жишээ команд бөгөөд энэ ажлын явцад заавал бүгдийг нь ажиллуулаагүй.

Arch Linux:

```
sudo pacman -S gcc make python
```

Debian / Ubuntu:

```
sudo apt install build-essential python3
```

Kernel module хөрвүүлэх нь заавал хийх алхам биш бөгөөд 8.2.7 хэсэгт тусад нь тайлбарласан. Хэрэв module хөрвүүлэх бол ажиллаж байгаа kernel-тэй тохирох header файл хэрэгтэй.

Arch Linux дээр:

```
sudo pacman -S linux-headers
```

Debian / Ubuntu дээр:

```
sudo apt install linux-headers-$(uname -r)
```

Манай туршилтын үед багийн гишүүн Arch Linux дээр `pacman` ашиглан kernel болон header-ийг суулгасан. Тухайн үед `linux` болон `linux-headers` хоёулаа `7.2.11.arch1-1` хувилбартай байсан.

2.4 Build ба Run командууд

Команд	Үр дүн
`make`	`build/kds_engine`, `build/libkds.a` (shim + engine core)
`make test`	Бүх тест (`build/sanitized/` дахь ASan+UBSan build-ээр). Алдаатай бол 0-ээс өөр кодтой гарна. SIGINT (Ctrl+C) үл тоомсорлогдсон орчинд (жишээ нь `nohup`, CI) ч ажиллана.
`make run`	UI сервер: `python3 src/ui/server.py --port 8080 --max-nodes 2048 --engine build/kds_engine`
`make module KDIR=…`	Жинхэнэ кернел модуль (Kbuild) `src/kernel/*.ko`. Кернелийн header шаардана (8.2.7). `KDIR` заагаагүй бол `/lib/modules/$(uname -r)/build` ашиглана.
`make clean`	`build/`, кэш болон кернелийн build үлдэгдлийг устгана. `src/kernel/*.ko` файлуудыг мөн устгана.

Engine-ийг UI ашиглахгүйгээр шууд ажиллуулж болно. Нэг команд нь нэг мөрөөр өгөгдөх бөгөөд хариу нь нэг мөр JSON хэлбэрээр буцна:

```
printf 'color example\ncolor delete index=1\nstate\n' | build/kds_engine
build/kds_engine --max-nodes 3 < tests/cases/max_nodes.in
```

2.5 Үндсэн тестийн өгөгдөл

Файл	Агуулга
`tests/test_list.c`	Өөрийн `list.h`: unit + 5,000 санамсаргүй үйлдэл (seed `0x0c011a72`)
`tests/test_color_module.c`, `test_collatz_module.c`	Жинхэнэ модулийн `.c` + shim: printk, leak, ENOMEM. Collatz 1, 15, 25, 27, 97, 837799, 2147483647
`tests/test_core.c`	Engine core, `kds_verify`-ийн 10 эвдэрсэн жагсаалт
`tests/cases/*.in` + `*.expected.json`	9 бүлэг: `book_example`, `color_operations`, `boundary_errors`, `stack_queue`, `collatz_reference`, `max_nodes`, `allocation_failure`, `ignored_lines`, `binary_garbage`
`tests/run_cases.py`	Сценарийн тест (`python3 tests/run_cases.py --engine build/kds_engine`)
`tests/test_server.py`	HTTP серверийн 12 тест. Үүнд процессын бүлэгт SIGINT илгээж цэвэр shutdown шалгах 2 тест орно: энгийн, мөн SIGINT-ийг «үл тоомсорлох» төлөвтэй өвлөсөн үед.
`tests/ui/browser_check.py`, `tests/ui/interactions.py`	UI шалгалт (Brave/Chromium, CDP): 4 таб, 112 зангилаа ба 226 сум, хайлт/устгал, RGB баталгаажуулалт, түүх, zoom/pan, горим, 320/390 px, JS алдаагүй. `make test`-д ороогүй.
`tests/README.md`	Тест бүрийн оролт, хүлээгдсэн үр дүн, гарал үүсэл

UI-ийн автомат шалгалтыг ажиллуулахын тулд эхлээд нэг терминал дээр:

```
make run
```

командыг ажиллуулж серверийг асаана.

Дараа нь өөр терминал нээгээд дараах хоёр скриптийг дарааллаар ажиллуулна:

```
python3 tests/ui/browser_check.py
```

```
python3 tests/ui/interactions.py
```

Эдгээр скриптүүд шалгалтын явцад engine-ийг дахин эхлүүлж, UI-ийн төлөвийг өөрчилж туршина. Шалгалтын үед авсан screenshot-ууд `/tmp/kds-ui-check` хавтсанд хадгалагдана.

Шаардлагатай бол дараах орчны хувьсагчдыг ашиглан тохируулж болно:

- `KDS_URL` — өөр серверийн хаяг ашиглах
- `KDS_BROWSER` — ашиглах browser-ийн замыг заах
- `KDS_OUTPUT` — screenshot болон бусад гаралтын файл хадгалах хавтсыг өөрчлөх

Тестийн скриптүүд нь зөвхөн локал, итгэмжит UI дээр ажиллуулах зориулалттай. Шалгалтын үед түр headless browser асаах бөгөөд `--no-sandbox` тохиргоог ашиглана.

Collatz лавлагааг дахин үүсгэх (Python, production кодыг ашиглахгүй): `python3 tests/reference_cases.py`.

2.6 Нэмэлт boundary/error тест ба хүлээгдсэн үр дүн

Engine-ийг UI ашиглахгүйгээр шууд шалгахдаа дараах хэлбэрийн командыг ашиглаж болно:

```
printf '...\n' | build/kds_engine
```

Алдаатай команд өгөх үед engine нь JSON хариунд `"ok": false` гэж буцааж, алдааны төрлийг `error.code` талбарт харуулна. Алдаа гарсан тохиолдолд өмнөх өгөгдлийн төлөв болон `kmalloc` санах ойн тоолуур өөрчлөгдөхгүй хэвээр үлдэнэ.

Команд	Хүлээгдсэн error.code
`color add_tail red=256 green=0 blue=0`; `red=-1` …	E_RANGE
`color add_tail red=abc green=0 blue=0`	E_BAD_VALUE
`color add_tail red=1 green=2`	E_MISSING_ARG
`color add_tail red=1 red=2 green=2 blue=3`	E_SYNTAX (давхардсан)
`color add_tail red=1 green=2 blue=3 alpha=4`	E_UNKNOWN_ARG
`color insert index=99 red=1 green=2 blue=3`; `color delete index=-1`	E_RANGE
`foo bar`	E_UNKNOWN_CMD
`color search red=1` (олдохгүй)	E_NOT_FOUND
`stack pop`, `stack peek`, `queue dequeue` (хоосон)	E_EMPTY
`collatz run start=0` / `-1` / `2147483648`	E_RANGE
`collatz run start=abc`; `collatz run`	E_BAD_VALUE; E_MISSING_ARG
600 тэмдэгттэй мөр	E_SYNTAX, дараагийн команд хэвийн

Команд	Хүлээгдсэн error.code
`--max-nodes 3` үед 4-р зангилаа	E_FULL
`inject_fail after=2` дараа 3-р `color add_tail …`	E_NOMEM; `inject_fail off` дараа амжилттай

Collatz-ийн баталгаажуулах утгууд: `collatz run start=N` командыг ажиллуулах үед дараах үр дүн гарах ёстой.

- `N = 1` → 1 зангилаа
- `N = 15` → 18 зангилаа
- `N = 25` → 24 зангилаа
- `N = 27` → 112 зангилаа, оргил утга `9232`
- `N = 97` → 119 зангилаа, оргил утга `9232`
- `N = 837799` → 525 зангилаа, оргил утга `2,974,984,576`
- `N = 2147483647` → 451 зангилаа, оргил утга `1,235,346,792,567,892`

Эдгээр утгуудыг тестийн үед баталгаажуулах үр дүн болгон ашигласан.

HTTP түвшин (сервер асаалттай; хүлээгдсэн статус нь бодитоор шалгагдсан):

```
U=http://127.0.0.1:8080
# 200, "ok":true
curl -s -X POST -H 'Content-Type: application/json' \
     -d '{"cmd":"color example"}' $U/api/cmd
# 400 E_SYNTAX
curl -s -X POST -H 'Content-Type: text/plain' -d '{"cmd":"state"}' $U/api/cmd
# 400 E_SYNTAX
curl -s -X POST -H 'Content-Type: application/json' -d '{bad' $U/api/cmd
# 400 (нэг мөр биш)
curl -s -X POST -H 'Content-Type: application/json' \
     -d '{"cmd":"state\nreset"}' $U/api/cmd
# 403 E_ORIGIN
curl -s -X POST -H 'Origin: http://evil.example' \
     -H 'Content-Type: application/json' -d '{"cmd":"state"}' $U/api/cmd
# 403
curl -s --path-as-is -o /dev/null -w '%{http_code}\n' "$U/../Makefile"
# 404
curl -s -o /dev/null -w '%{http_code}\n' $U/nonexistent.js
# 200, бүх бүтэц хоосон
curl -s -X POST $U/api/restart
```

2.7 Жинхэнэ кернел модуль (суулгах, ачаалах, зайлуулах)

Доорх алхмуудыг багийн гишүүн өөрийн Arch Linux машин дээр, `Linux 7.2.11-arch1-1` kernel ашиглан өөрөө ажиллуулж шалгасан.

Kernel module-ийг шууд ажиллаж байгаа kernel-д ачаалдаг тул буруу module эсвэл ноцтой алдаа гарсан тохиолдолд системийн ажиллагаанд нөлөөлөх эрсдэлтэй. Тиймээс module ачаалахаас өмнө нээлттэй ажлуудаа хадгалах хэрэгтэй. Боломжтой бол эхний туршилтыг виртуал машин дээр хийх нь илүү аюулгүй.

Мөн Secure Boot эсвэл module signing идэвхтэй орчинд гарын үсэггүй `.ko` module ачаалагдахгүй байх боломжтой. Энэ нөхцөлийг тусад нь шалгаагүй. Манай багийн гишүүний машин дээр module-ууд нэмэлт тохиргоогүйгээр амжилттай ачаалагдсан.

1. Kernel header суулгах

Module хөрвүүлэхийн тулд ашиглаж байгаа kernel-тэй таарах header хэрэгтэй. Arch Linux дээр:

```
sudo pacman -S linux-headers
```

Header зөв суусан эсэхийг дараах командаар шалгаж болно:

```
ls /usr/lib/modules/$(uname -r)/build
```

Хэрэв энэ хавтас байгаа бол module хөрвүүлэхэд шаардлагатай header бэлэн гэсэн үг.

2. Module хөрвүүлэх

Дараах командыг ажиллуулна:

```
make module KDIR=/usr/lib/modules/$(uname -r)/build
```

Амжилттай хөрвүүлэгдсэн тохиолдолд хоёр module дээр `CC [M]`, дараа нь `MODPOST`, `LD [M]`, `BTF [M]` гэсэн алхмууд харагдана. Хөрвүүлэлтийн үед алдаа болон warning гарахгүй байх ёстой.

`src/kernel/` хавтсанд дараах хоёр файл үүснэ:

- `color_list.ko` — манай туршилтаар 213,424 байт
- `collatz.ko` — манай туршилтаар 359,088 байт

Файлын хэмжээ compiler болон build-ийн замаас шалтгаалан бага зэрэг өөр байж болно.

`modinfo` командаар шалгахад module-ийн license нь `GPL`, vermagic нь `7.2.11-arch1-1 SMP preempt mod_unload` гэж гарсан. `collatz` module нь мөн `start` гэсэн `int` параметртэй.

3. Part I — өнгөний module

Эхлээд module-ийг ачаална:

```
sudo insmod src/kernel/color_list.ko
```

Дараа нь kernel log-ийг шалгана:

```
sudo dmesg | tail -20
```

Туршилт дууссаны дараа module-ийг салгана:

```
sudo rmmod color_list
```

`dmesg` дээр нийт 4 өнгө хэвлэгдэх бөгөөд эхний мөр нь violet байна:

```
color[0]: red=138, green=43, blue=226
```

```
color[1]: red=255, green=0, blue=0
```

```
color[2]: red=0, green=255, blue=0
```

```
color[3]: red=0, green=0, blue=255
```

4. Part II — Collatz module

Collatz module-ийг `start=27` параметртэй ачаална:

```
sudo insmod src/kernel/collatz.ko start=27
```

Дараа нь гаралтыг харна:

```
sudo dmesg | tail -130
```

Туршилтын дараа module-ийг салгана:

```
sudo rmmod collatz
```

`start=27` үед нийт 112 мөр гарах ёстой бөгөөд индекс нь `0`-оос `111` хүртэл байна. Дараалал `27, 82, 41, 124, ...` гэж эхэлж, `77`-р индекс дээр хамгийн их утга болох `9232` гарна. Төгсгөл хэсэг нь `4, 2, 1` байна.

Жишээ:

```
collatz[0]: value=27

collatz[1]: value=82

...

collatz[77]: value=9232

...

collatz[111]: value=1
```

Бүх 112 мөрийг харахын тулд `tail -130` ашигласан. Үүнээс шалтгаалж өмнөх kernel log-ийн хэдэн мөр хамт харагдаж болно.

2.8 Хүлээгдсэн `make test` үр дүн

Төслийн цэвэр хуулбар дээр `make && make test` командыг ажиллуулахад процесс `exit code 0`-тэй амжилттай дууссан. Манай туршилтаар нийт хугацаа ойролцоогоор 2.4–2.5 секунд байсан.

Тестийн гол үр дүнгээс:

```
Sanitizers: ASan + UBSan

PASS real Collatz module: default 25, 7 references, 1250 injected
failures, 2 invalid starts; zero leaks

PASS real color module: exact printk, 4 colors, violet RGB, 4 ENOMEM
positions + success boundary, no leaks

PASS core commands, IDs, atomic errors, limits, ENOMEM sweep,
512-byte boundary, 10 corrupt lists
```

```
PASS list API + seeded differential model + zero leaks

CASES: 9 passed, 0 failed; 229 checked replies

Ran 12 tests in 0.391s

OK

All available tests passed.
```

Эдгээрээс харахад Collatz болон өнгөний module-ийн логик, санах ойн алдаа, буруу оролт, жагсаалтын бүтцийн шалгалт, engine-ийн үндсэн командууд болон HTTP түвшний тестүүд бүгд амжилттай болсон.

`0.391 s` гэх мэт ажилласан хугацаа болон санах ойн хаягууд нь компьютер бүр дээр бага зэрэг өөр байж болно. Мөн `make test` нь kernel header шаарддаггүй бөгөөд kernel module-ийг жинхэнэ kernel-д ачаалдаггүй.

2.9 Зарим Засварлах Дутагдлууд

- **Shim-ийн хязгаарлалт:** Shim нь жинхэнэ kernel орчны бүх нөхцөлийг дуурайдаггүй. Жишээлбэл `GFP` флаг, SMP орчин болон санах ойн дарамтыг загварчлахгүй. `ENOMEM` алдааг тестийн зориулалтаар зориудаар үүсгэдэг.
- **Server тест:** `tests/test_server.py` нь Linux-ийн `/proc/net/tcp` файлыг ашигладаг. Тиймээс macOS болон Windows орчинд энэ тест ажиллахгүй байж болно.
- **Порт:** `make run` нь анхдагчаар `8080` портыг ашиглана. Хэрэв өөр порт хэрэгтэй бол серверийг шууд параметртэй ажиллуулж болно. Үүнийг 8.1.4 хэсэгт тайлбарласан. Хэрэв `8080` порт аль хэдийн ашиглагдаж байгаа бол Python сервер асахгүй бөгөөд терминал дээр алдааны мэдээлэл гарна.
- **Collatz delete:** `collatz delete` үйлдлээр дарааллын дурын зангилааг устгаж болно. Үүний дараа үлдсэн жагсаалт Collatz-ийн математик дүрмийг заавал дагахгүй. Энэ үйлдэл нь Collatz дарааллыг дахин тооцоолоход биш, холбоост жагсаалтаас элемент устгах ажиллагааг харуулах зориулалттай.
- **Санах ойн дугаар:** Амжилтгүй командын үед түр авсан санах ойг буцааж чөлөөлдөг. Тиймээс UI дээр харагдах `kmalloc`-ийн дугааруудын хооронд заримдаа алгасалт гарч болно.
- **Серверийн хэрэглээ:** Сервер нь зөвхөн тухайн компьютер дээр `127.0.0.1` хаягаар ажиллах зориулалттай. Олон хэрэглэгч зэрэг ашиглах production сервер байдлаар хийгдээгүй.